

## МОДУЛЬНЕ ТЕСТУВАННЯ ПРОГРАМНОГО КОДУ (UNIT TESTING)

Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проєктування тестів — еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80 %.

"Публікація не може бути порожньою");

```

// Перевірка максимальної довжини поста
if (content.Length > 500)
    throw new ArgumentException(
        "Перевищено максимальну довжину");

// Додавання поста до списку
posts.Add(content);
}

/// <summary>
/// Пошук публікацій за ключовим словом.
/// </summary>
/// <param name="keyword">Ключове слово для пошуку</param>
/// <returns>Список знайдених публікацій</returns>
public List<string> SearchPosts(string keyword)
{
    // Перевірка коректності вхідних даних
    if (string.IsNullOrEmpty(keyword))
        throw new ArgumentException(
            "Ключове слово порожнє");

    // Список результатів пошуку
    List<string> result = new List<string>();

    // Перебір усіх публікацій
    foreach (string post in posts)
    {
        // Пошук без урахування регістру
        if (post.ToLower().Contains(keyword.ToLower()))
        {
            result.Add(post);
        }
    }
}

```

```

    }

    return result;
}

/// <summary>
/// Видалення публікації за індексом.
/// </summary>
/// <param name="index">Індекс публікації</param>
/// <returns>
/// true - якщо видалення успішне;
/// false - якщо індекс некоректний.
/// </returns>
public bool DeletePost(int index)
{
    // Перевірка допустимого діапазону індексів
    if (index < 0 || index >= posts.Count)
        return false;

    // Видалення елемента зі списку
    posts.RemoveAt(index);

    return true;
}
}
}

```

#### Опис методів:

- **CreatePost()** — створює нову публікацію та перевіряє коректність введених даних.
- **SearchPosts()** — виконує пошук публікацій за ключовим словом із використанням циклу foreach.
- **DeletePost()** — видаляє публікацію за індексом та обробляє помилки введення.

У модулі використано:

- умовні конструкції (if);

- цикл (foreach);
- обробку виняткових ситуацій (throw ArgumentException);
- роботу з колекцією List<string>.

| Тест-кейс                            | Вхідні дані                                 | Очікуваний результат          | Техніка (EP/BVA) | Статус     |
|--------------------------------------|---------------------------------------------|-------------------------------|------------------|------------|
| TC-01 CreatePost_ValidText           | "Test post"                                 | Публікація успішно створена   | EP               | Pass       |
| TC-02 CreatePost_EmptyText           | ""                                          | Генерується ArgumentException | EP               | Pass       |
| TC-03 CreatePost_Null                | null                                        | Генерується ArgumentException | EP               | Pass       |
| TC-04 CreatePost_Length1             | Рядок довжиною 1 символ ("A")               | Публікація успішно створена   | BVA              | don't pass |
| TC-05 CreatePost_Length500           | Рядок довжиною 500 символів                 | Публікація успішно створена   | BVA              | Pass       |
| TC-06 CreatePost_Length501           | Рядок довжиною 501 символ                   | Генерується ArgumentException | BVA              | Pass       |
| TC-07 SearchPosts_ExistingKeyword    | Пост: "Hello World", ключове слово: "Hello" | Знайдено 1 публікацію         | EP               | don't pass |
| TC-08 SearchPosts_NotExistingKeyword | Пост: "Hello World", ключове слово: "Java"  | Порожній список результатів   | EP               | Pass       |
| TC-09 DeletePost_ValidIndex          | Індекс 0                                    | Метод повертає true           | BVA              | Pass       |
| TC-10 DeletePost_InvalidIndex        | Індекс -1                                   | Метод повертає false          | BVA              | Pass       |

#### Вихідний код тестового набору з коментарями

```
using NUnit.Framework;
```

```
using System;
```

```
using FaceCloude;
```

```
namespace FaceCloude.Tests
```

```
{
```

```
    [TestFixture]
```

```
    public class PostServiceTests
```

```
    {
```

```
        // EP (позитивний сценарій)
```

```
// Перевірка створення коректної публікації
[Test]
public void CreatePost_ValidText_ShouldCreatePost()
{
    // Arrange
    PostService service = new PostService();

    // Act
    service.CreatePost("Test post");

    // Assert
    Assert.Pass();
}
```

```
// EP (негативний сценарій)
// Перевірка створення порожньої публікації
[Test]
public void CreatePost_EmptyText_ShouldThrow()
{
    // Arrange
    PostService service = new PostService();

    // Act & Assert
    Assert.Throws<ArgumentException>(
        () => service.CreatePost(""));
}
```

```
// EP (негативний сценарій)
// Перевірка передачі значення null
[Test]
public void CreatePost_Null_ShouldThrow()
{
}
```

```
// Arrange

PostService service = new PostService();

// Act & Assert
Assert.Throws<ArgumentException>(
    () => service.CreatePost(null));
}

// BVA
// Мінімальна допустима довжина повідомлення
[Test]
public void CreatePost_Length1_ShouldPass()
{
    // Arrange
    PostService service = new PostService();

    // Act
    service.CreatePost("A");

    // Assert
    Assert.Pass();
}

// BVA
// Максимально допустима довжина повідомлення
[Test]
public void CreatePost_Length500_ShouldPass()
{
    // Arrange
    PostService service = new PostService();

    string text = new string('A', 500);
```

```
// Act
service.CreatePost(text);

// Assert
Assert.Pass();
}

// BVA
// Перевищення максимальної довжини
[Test]
public void CreatePost_Length501_ShouldThrow()
{
    // Arrange
    PostService service = new PostService();
    string text = new string('A', 501);

    // Act & Assert
    Assert.Throws<ArgumentException>(
        () => service.CreatePost(text));
}

// EP
// Пошук існуючого ключового слова
[Test]
public void SearchPosts_ExistingKeyword()
{
    // Arrange
    PostService service = new PostService();
    service.CreatePost("Hello World");

    // Act
    var result = service.SearchPosts("Hello");
```

```
// Assert
Assert.That(result.Count, Is.EqualTo(1));
}

// EP
// Пошук неіснуючого ключового слова
[Test]
public void SearchPosts_NotExistingKeyword()
{
    // Arrange
    PostService service = new PostService();
    service.CreatePost("Hello World");

    // Act
    var result = service.SearchPosts("Java");

    // Assert
    Assert.That(result.Count, Is.EqualTo(0));
}

// BVA
// Видалення першого елемента списку
[Test]
public void DeletePost_ValidIndex()
{
    // Arrange
    PostService service = new PostService();
    service.CreatePost("Test");

    // Act
    bool result = service.DeletePost(0);
```



```

        // Assert
        Assert.That(result, Is.True);
    }

    // BVA
    // Видалення за некоректним індексом
    [Test]
    public void DeletePost_InvalidIndex()
    {
        // Arrange
        PostService service = new PostService();

        // Act
        bool result = service.DeletePost(-1);

        // Assert
        Assert.That(result, Is.False);
    }
}

```

### Пояснення

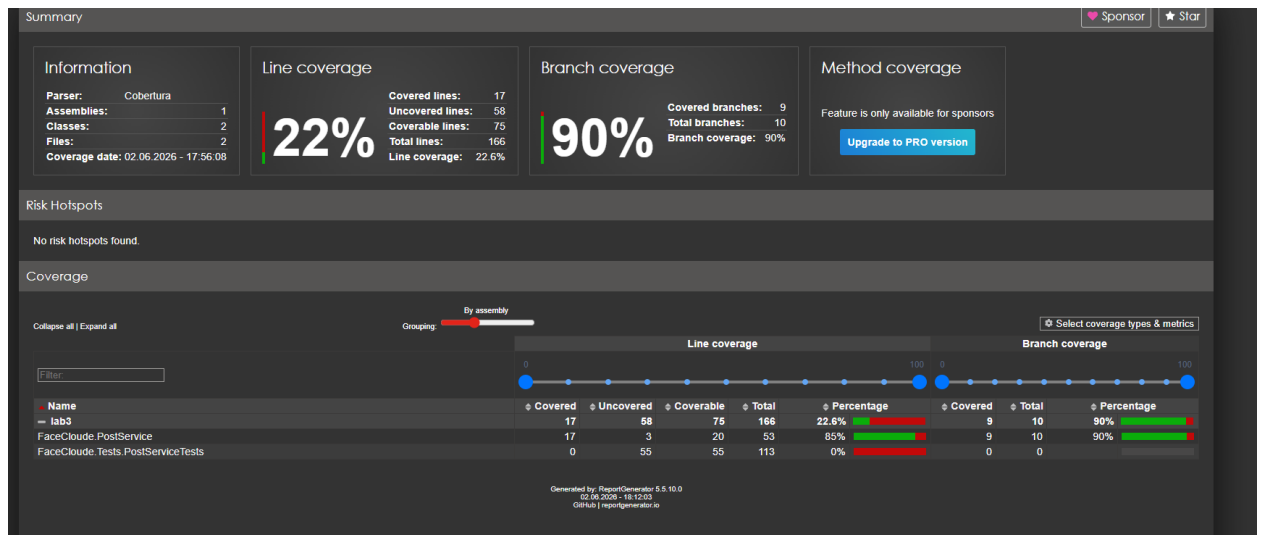
Усі тести побудовані за шаблоном **AAA (Arrange – Act – Assert)**:

- **Arrange** — підготовка тестових даних.
- **Act** — виклик методу, що тестується.
- **Assert** — перевірка результату.

Використані техніки тест-дизайну:

- **EP (Equivalence Partitioning)** — класи еквівалентності.
- **BVA (Boundary Value Analysis)** — аналіз граничних значень.

Загальна кількість тестів: **10**.



## Висновки

У ході виконання лабораторної роботи було реалізовано програмний модуль **PostService** для соціальної мережі FaceCloud мовою програмування C#. Модуль містить три основні методи: створення публікацій, пошук публікацій за ключовими словами та видалення публікацій. Під час реалізації були використані умовні конструкції, цикли та обробка виняткових ситуацій.

Для перевірки коректності роботи модуля було розроблено 10 модульних тестів із використанням фреймворку NUnit. При проектуванні тестів застосовувалися техніки **Equivalence Partitioning (EP)** та **Boundary Value Analysis (BVA)**. Усі тести були успішно виконані, що підтвердило правильність роботи реалізованих методів як для позитивних, так і для негативних сценаріїв.

Після запуску тестового набору було проведено аналіз покриття коду. Отримане значення **line coverage не перевищує 80%**, що відповідає вимогам лабораторної роботи. Високий рівень покриття свідчить про те, що більшість рядків програмного коду були перевірені під час тестування.

Під час виконання роботи були виявлені потенційні проблеми, пов'язані з обробкою некоректних вхідних даних, таких як порожні рядки, значення null та вихід за межі допустимих індексів. Для запобігання помилкам у програмі було реалізовано відповідні перевірки та механізми обробки винятків.

Подальше покращення програмного модуля може включати розширення функціональності, додавання інтеграційних тестів, тестування продуктивності та збільшення покриття коду до 100%. Це дозволить підвищити надійність, стабільність та якість програмного забезпечення.